

Java Interview

Practice Question Bank

Multithreading · Exception Handling · Java 8 · Collections

Easy → Medium → Hard · Theory + Coding · Salary Insights

■■ No solutions are provided. This is a pure practice sheet. Solve each problem independently, then verify using documentation or an IDE.

Contents

1. Multithreading — 21 Coding + 15 Theory Questions
2. Exception Handling — 18 Coding + 14 Theory Questions
3. Java 8 Features — 23 Coding + 15 Theory Questions
4. Collections Framework — 23 Coding + 15 Theory Questions
5. Salary Expectations Guide

Chapter 1: Multithreading

Concurrency, Synchronization & Thread Management

CODING QUESTIONS

● Easy Level

Q1.

Create a thread by extending Thread class and print numbers 1-10.

Hint: Override run() method

Q2.

Create a thread using Runnable interface and print even numbers.

Hint: Pass to Thread constructor

Q3.

Start two threads simultaneously and print their names.

Hint: Use Thread.currentThread().getName()

Q4.

Make the main thread wait for a child thread to finish.

Hint: Look into join()

Q5.

Write a program to demonstrate thread sleep.

Hint: Use Thread.sleep(ms)

Q6.

Print numbers 1-5 using three different threads, each printing one number.

Hint: Use thread constructor with lambda

Q7.

Demonstrate the use of isAlive() method.

Hint: Check before and after join()

● Medium Level

Q8.

Implement a producer-consumer problem using wait() and notify().

Hint: Use a shared buffer with synchronized block

Q9.

Write a program where two threads increment a shared counter to 1000 and show the race condition, then fix it using synchronized.

Hint: Observe wrong result first, then fix

Q10.

Implement a thread-safe Singleton using double-checked locking.

Hint: Use volatile + synchronized

Q11.

Create a thread pool of 4 threads to execute 10 tasks using `ExecutorService`.

Hint: `Executors.newFixedThreadPool(4)`

Q12.

Implement a simple deadlock between two threads and then resolve it.

Hint: Lock ordering is the fix

Q13.

Use `Callable` and `Future` to compute the factorial of a number in a separate thread.

Hint: `submit()` returns `Future`

Q14.

Write a program using `CountDownLatch` where 3 worker threads signal a main thread.

Hint: `countdown()` and `await()`

Q15.

Implement a bounded blocking queue using `ReentrantLock` and `Condition`.

Hint: `lock.newCondition()`

● Hard Level

Q16.

Implement a thread-safe LRU cache using `ConcurrentHashMap` and `LinkedHashMap`.

Hint: Override `removeEldestEntry`

Q17.

Design a custom thread pool from scratch without using `ExecutorService`.

Hint: `BlockingQueue` + worker threads

Q18.

Implement the dining philosophers problem with deadlock prevention.

Hint: Resource hierarchy or Chandy/Misra

Q19.

Write a program using `CyclicBarrier` where 5 threads simulate parallel matrix row computation.

Hint: `barrier.await()` at each phase

Q20.

Implement a rate limiter using semaphores (allow N requests per second).

Hint: `Semaphore.tryAcquire(timeout)`

Q21.

Use `ForkJoinPool` to parallelize merge sort.

Hint: Extend `RecursiveTask`

Q22.

Implement a read-write lock from scratch using `ReentrantLock`.

Hint: Separate read/write counts

THEORY / INTERVIEW QUESTIONS

- T1.** What is the difference between a process and a thread?
- T2.** What are the different states in a Thread lifecycle?
- T3.** What is the difference between synchronized method and synchronized block?
- T4.** Explain the volatile keyword. When would you use it?
- T5.** What is a race condition and how do you prevent it?
- T6.** What is deadlock? What are the four necessary conditions?
- T7.** Difference between wait(), sleep(), and yield()?
- T8.** What is ThreadLocal? Give a real-world use case.
- T9.** What is the difference between Callable and Runnable?
- T10.** What is the Java Memory Model (JMM) and how does it affect concurrency?
- T11.** Explain happens-before relationship in Java concurrency.
- T12.** What is ConcurrentHashMap and how is it different from Hashtable?
- T13.** What are atomic classes (AtomicInteger etc.) and when to use them?
- T14.** Explain the Fork/Join framework and work-stealing algorithm.
- T15.** What is starvation and livelock? How are they different from deadlock?

Chapter 2: Exception Handling

Checked, Unchecked Exceptions & Best Practices

CODING QUESTIONS

● Easy Level

Q1.

Write a program that catches an `ArrayIndexOutOfBoundsException` and prints a friendly message.

Hint: Use try-catch block

Q2.

Demonstrate finally block — show it always executes even when exception is thrown.

Hint: Put cleanup code in finally

Q3.

Write a method that throws a checked exception (`IOException`) and handle it in main.

Hint: Use throws declaration

Q4.

Create a custom exception class `InvalidAgeException` and throw it if age < 0.

Hint: Extend Exception class

Q5.

Catch multiple exceptions in a single catch block (multi-catch).

Hint: Exception1 | Exception2 syntax

Q6.

Demonstrate try-with-resources using a `BufferedReader`.

Hint: Implements AutoCloseable

● Medium Level

Q7.

Write a program where a custom checked exception is chained to another exception.

Hint: Use `initCause()` or constructor

Q8.

Implement a method that reads a file and throws a custom `FileParseException` if format is invalid.

Hint: Wrap `IOException` in custom

Q9.

Write a recursive method with exception handling — prevent `StackOverflowError` gracefully.

Hint: Catch Error type

Q10.

Demonstrate exception propagation through multiple method calls — show stack trace.

Hint: Let it bubble up, catch at top

Q11.

Implement retry logic: retry an operation 3 times before giving up, using exceptions.

Hint: Loop with catch + counter

Q12.

Create a hierarchy of custom exceptions (AppException > DatabaseException > ConnectionException).

Hint: Catch parent to handle all

Q13.

Demonstrate difference: throw vs throws vs Throwable.

Hint: Write one example for each

● Hard Level

Q14.

Design a global exception handler for a REST API simulation (no frameworks).

Hint: Wrap all calls, log and rethrow

Q15.

Write a program where finally block has a return statement — explain the output.

Hint: finally return overrides try return

Q16.

Implement a circuit-breaker pattern using exception counting.

Hint: Track failures, open/close state

Q17.

Demonstrate exception handling in multi-threaded code — uncaught exception handler.

Hint: Thread.setDefaultUncaughtExceptionHandler

Q18.

Build a validation framework that collects all validation errors before throwing.

Hint: Accumulate in list, throw once

THEORY / INTERVIEW QUESTIONS

T1. What is the difference between Error and Exception?

T2. What is the difference between checked and unchecked exceptions?

T3. Can we catch an Error? Should we?

T4. What happens if an exception is thrown inside a finally block?

T5. What is exception chaining and why is it useful?

T6. What is try-with-resources and what interface must be implemented?

T7. Can a catch block throw an exception?

T8. What is the difference between throw and throws?

T9. Explain the exception hierarchy in Java.

T10. What are best practices for creating custom exceptions?

T11. What is the impact of exception handling on performance?

T12. Can we have a try block without catch?

T13. What is a suppressed exception (introduced in Java 7)?

T14. How do you handle exceptions in streams (Java 8)?

Chapter 3: Java 8 Features

Lambdas, Streams, Optional & Functional Interfaces

CODING QUESTIONS

● Easy Level

Q1.

Use a lambda to sort a list of strings alphabetically.

Hint: Comparator.comparing()

Q2.

Filter a list of integers to get only even numbers using Stream.filter().

Hint: stream().filter(n -> n%2==0)

Q3.

Use map() to convert a list of strings to uppercase.

Hint: stream().map(String::toUpperCase)

Q4.

Use Optional to avoid NullPointerException when finding a name in a list.

Hint: findFirst().orElse()

Q5.

Write a method reference equivalent for String::length.

Hint: Instance method reference

Q6.

Use Collectors.toList() to collect stream results.

Hint: Terminal operation

Q7.

Create a Predicate that checks if a string starts with 'A'.

Hint: Functional interface

● Medium Level

Q8.

Given a list of employees, find the average salary using streams.

Hint: mapToDouble().average()

Q9.

Group a list of students by their grade using Collectors.groupingBy().

Hint: Returns Map>

Q10.

Write a program to find the 3 most expensive products using streams.

Hint: sorted().limit(3)

Q11.

Use `flatMap()` to flatten a list of lists into a single list.

Hint: `flatMap(Collection::stream)`

Q12.

Compose two Predicates using `and()`, `or()`, `negate()`.

Hint: `pred1.and(pred2)`

Q13.

Use `reduce()` to find the product of all numbers in a list.

Hint: `reduce(1, (a,b) -> a*b)`

Q14.

Implement a custom Comparator using lambda to sort employees by salary then name.

Hint: `thenComparing()`

Q15.

Use `Collectors.partitioningBy()` to split a list into passing/failing students.

Hint: Returns Map

Q16.

Chain Optional operations: `map`, `filter`, `orElseThrow`.

Hint: No null checks needed

● Hard Level

Q17.

Build a data pipeline: read a CSV-like list, parse, filter, transform, and collect statistics in one stream chain.

Hint: Multi-step stream pipeline

Q18.

Implement your own functional interface with a default method and use it with lambda.

Hint: `@FunctionalInterface` annotation

Q19.

Use parallel streams to compute the sum of squares of 1M numbers — compare with sequential.

Hint: `parallelStream()` vs `stream()`

Q20.

Use `Collectors.toMap()` to build a Map of word frequencies from a text.

Hint: Handle duplicate key with merge function

Q21.

Implement memoization of a Function using a Map.

Hint: `computeIfAbsent()`

Q22.

Write a lazy infinite stream using `Stream.iterate()` and `limit()`.

Hint: Fibonacci with `iterate()`

Q23.

Use `CompletableFuture.supplyAsync()` to fetch two results in parallel and combine them.

Hint: `thenCombine()`

THEORY / INTERVIEW QUESTIONS

- T1.** What is a lambda expression? What problem does it solve?
- T2.** What is a functional interface? Name 4 built-in ones.
- T3.** What is the difference between `map()` and `flatMap()`?
- T4.** What are intermediate vs terminal stream operations? Give examples.
- T5.** What is `Optional` and why was it introduced?
- T6.** What is the difference between `findFirst()` and `findAny()`?
- T7.** Explain method references — 4 types with examples.
- T8.** What is the difference between `Comparator.comparing()` and `thenComparing()`?
- T9.** What are default and static methods in interfaces?
- T10.** When should you use parallel streams? What are the pitfalls?
- T11.** What is the difference between `reduce()` and `collect()`?
- T12.** Explain the difference between `peek()` and `forEach()`.
- T13.** What is `CompletableFuture` and how is it better than `Future`?
- T14.** What are the new Date/Time API classes introduced in Java 8?
- T15.** What is the difference between `stream()` and `parallelStream()` thread-safety?

Chapter 4: Collections Framework

Lists, Maps, Sets, Queues & Generics

CODING QUESTIONS

● Easy Level

Q1.

Add, remove, and iterate over an ArrayList.

Hint: Basic CRUD with index

Q2.

Demonstrate the difference between ArrayList and LinkedList for add/get operations.

Hint: Time it or explain Big-O

Q3.

Use a HashSet to remove duplicates from a list of integers.

Hint: new HashSet<>(list)

Q4.

Store key-value pairs in a HashMap and retrieve by key.

Hint: put() and get()

Q5.

Sort a List using Collections.sort() and Comparator.

Hint: Natural order vs custom

Q6.

Use a Queue (LinkedList) to implement FIFO processing.

Hint: offer() and poll()

Q7.

Convert an array to List and back.

Hint: Arrays.asList() and toArray()

● Medium Level

Q8.

Implement a frequency counter using Map for words in a sentence.

Hint: getOrDefault() pattern

Q9.

Write a program to find the first non-repeating character using LinkedHashMap.

Hint: Insertion-order preserved

Q10.

Use TreeMap to display student marks in sorted order of name.

Hint: Natural sort of keys

Q11.

Implement a stack using Deque (ArrayDeque).

Hint: push() / pop()

Q12.

Compare performance: HashMap vs TreeMap vs LinkedHashMap — when to use each?

Hint: Write test with 10k entries

Q13.

Use PriorityQueue to find the K largest elements from a stream.

Hint: Min-heap of size K

Q14.

Implement a simple cache using LinkedHashMap with access-order (LRU).

Hint: new LinkedHashMap<>(cap, 0.75f, true)

Q15.

Write a generic method that works with any Collection type.

Hint: Use bounded wildcards

Q16.

Deep copy a List<> (2D list).

Hint: Nested iteration or stream

● Hard Level

Q17.

Design a multi-map (Map<>) from scratch without external libraries.

Hint: computeIfAbsent pattern

Q18.

Find the top-N frequent elements using HashMap + PriorityQueue.

Hint: Bucket sort or heap approach

Q19.

Implement an immutable collection wrapper class.

Hint: Delegate + throw on mutate

Q20.

Write a thread-safe counter map using ConcurrentHashMap.merge().

Hint: merge(key, 1, Integer::sum)

Q21.

Flatten a nested Map<> into a single-level map.

Hint: entrySet().stream().flatMap

Q22.

Implement a Trie data structure using Map.

Hint: Map-based children

Q23.

Design a bidirectional map (key->value and value->key lookup) using two HashMaps.

Hint: Keep both in sync on put/remove

THEORY / INTERVIEW QUESTIONS

- T1.** What is the difference between Collection and Collections?
- T2.** Explain the hierarchy of the Collections Framework.
- T3.** What is the difference between List, Set, and Queue?
- T4.** How does HashMap work internally (hashing, buckets, chaining)?
- T5.** What is the load factor and initial capacity of HashMap?
- T6.** What happens when two keys have the same hashCode?
- T7.** What is the difference between HashMap and Hashtable?
- T8.** Why should you override both hashCode() and equals()?
- T9.** What is the difference between fail-fast and fail-safe iterators?
- T10.** When would you use TreeMap over HashMap?
- T11.** What is the difference between Comparable and Comparator?
- T12.** What is CopyOnWriteArrayList and when should you use it?
- T13.** Explain the internal working of LinkedHashMap.
- T14.** What is the difference between poll() and remove() in Queue?
- T15.** What are wildcards (?, extends, super) in generics?

Chapter 5: Salary Expectations

What package can you expect after mastering these topics?

Solving the problems in this booklet — especially medium and hard levels — prepares you for real-world Java interviews. Here is a realistic salary guide based on topic mastery and experience (India market, 2024-25).

By Experience & Skill Level

Experience Level	Skills Mastered	Expected Package (India)
Fresher (0-1 yr)	Java 8 + Collections + Exceptions	4 – 8 LPA
Junior (1-3 yrs)	Above + Multithreading basics	8 – 15 LPA
Mid-Level (3-5 yrs)	All 4 topics + Spring Boot	15 – 28 LPA
Senior (5-8 yrs)	Deep concurrency + system design	28 – 50 LPA
Lead / Architect (8+)	Full mastery + mentoring	50 – 90 LPA

By Company Tier

Company Tier	Typical Range (India)	Focus Areas in Interview
Tier 1 (FAANG/MNC)	30 – 90+ LPA	Data structures, concurrency, system design
Tier 2 (Mid-size product)	15 – 40 LPA	Java internals, Spring, microservices
Service companies (TCS/Infy)	4 – 18 LPA	Core Java, basics, frameworks
Startups	10 – 35 LPA	Practical problem solving, speed

■ Salary Boosting Tips

- Mastering all Easy questions → Junior/Fresher roles (4-10 LPA)
- Solving Medium consistently → Mid-level roles (12-25 LPA)
- Cracking Hard questions → Senior/FAANG roles (30+ LPA)
- Combine with Spring Boot, SQL, and System Design for top packages
- Open-source contributions and GitHub profile greatly boost offers